

Submission:

-You are required to use Visual Studio for the assignment. Combine all your work (solution folder) in one .zip file after performing "Clean Solution". Name the .zip file as ROLLNUMBER-A2.zip (e.g. F2024-2345-A2.zip).

-Submit zip file on Google classroom within given deadline.

-Failure to submit according to above format would result in deduction of 10% marks.

Deadline: Deadline to submit assignment is **2nd May, 2025, 10 AM**. No submission will be considered for grading outside Google classroom or after this deadline. Correct and timely submission of assignment is responsibility of every student; hence no relaxation will be given to anyone.

Plagiarism: 0 marks will be given if any part of the assignment is found plagiarized. A code is considered plagiarized if more than 20% code is not your own work.

Mode of Testing: Viva will be conducted by the TA throughout the week. The schedule will be shared after the submission deadline.

Scenario: Stacks are not just for reversing strings or checking parentheses. Imagine a world where every action you take is recorded — and you have the ability to undo and redo actions at will. This is the core idea behind many real-life systems: text editors, web browsers, game states, time-travel debugging, and even your own brain's decision-making.

In this assignment, you will simulate a personalized Undo/Redo System, track your behavioral decisions across time, and use that to predict future decision patterns — all using custom stack implementations. You are required to completed the following three taasks:

Part 1: MyDailyStack

You will implement your own stack class from scratch (no built-in STL containers allowed). This stack must support the following operations:

- push(Action): Add a new action to the stack.
- pop(): Undo the most recent action.
- peek(): See the most recent action without undoing it.
- isEmpty(): Check if any actions exist.
- print(): Print the full stack in reverse order of actions.

Each action must be represented using a class, Action, as shown below.

```
class Action {
private:
    string type;        // e.g., "Study", "Play", "Procrastinate"
    string timeStamp; // e.g., "10:30AM"
};
```

Input Requirement:

You are required to read all action data from a text file named **full_actions.txt**.

Each line in the file will follow this format:

Study,9:00AM

Play,10:00AM

Procrastinate,11:00AM

...

Your program should read all actions for 90 days, assuming 12 actions per day (one per hour from 9AM to 9PM), and push them into a stack for each day.

Part 2: Undo/Redo Simulator

Implement a system using two stacks:

- Undo Stack – keeps track of actions.
- Redo Stack – stores undone actions.

Operations:

- performAction(Action): pushes to Undo, clears Redo.
- undo(): pops from Undo and pushes to Redo.
- redo(): pops from Redo and pushes back to Undo.

Simulate one full day from your logs and perform a random sequence of 50 perform, undo, and redo operations. Visualize both stacks after each operation. Test cases are provided to understand how Undo and Redo stacks are updated.

Part 3: PredictMyDay – Pattern Analysis with Stack History

You will now predict your most likely next action based on recent patterns.

1. Extract the last 7 days of action sequences from the Undo stack.
2. Convert each day's actions into a "signature pattern" string, e.g.:
"Study->Study->Play->Procrastinate->Study"
3. Compare your current week to past weeks using a similarity score:
 - +2 if same action in same time slot
 - +1 if action occurs but in a different slot
 - -1 if action is missing
4. Pick the most similar week, and assign the most common action in the 9AM–10AM slot from that week as your prediction for tomorrow.